Tài liệu bổ trợ

Bài tập lớn hợp nhất — Toán tử điểm, toán tử lân cận và phép biến đổi affine

CLAHE • Ngưỡng thích nghi bằng ảnh tích phân • Hough đa tỉ lệ

Môn Thị giác máy tính — Mã môn 121036
Trường Đại học Giao thông vận tải TP.HCM

Tài liệu này dùng để làm gì?

Ba kỹ thuật trình bày ở đây *không được giảng chi tiết trên lớp*. Chúng nằm vừa đúng một bước bên ngoài nội dung bài giảng: mỗi kỹ thuật chỉ dùng những viên gạch các em đã có (bảng tra LUT, tích chập, nội suy song tuyến, tổng tích lũy, không gian tham số Hough), nhưng ghép lại theo một cách mà bài giảng chưa nói tới. Tài liệu cung cấp **đủ lý thuyết để tự cài đặt**, **đặc tả hàm cần nộp**, **yêu cầu thí nghiệm minh họa** và **danh sách lỗi thường gặp**.

Mỗi kỹ thuật gồm hai phần nộp: **phần cài đặt** (tự viết, *không* dùng AI) và **phần thí nghiệm** (được dùng AI hỗ trợ, kèm prompt). Xem mục 5.1.

Tài liệu này *không* phải là Giải pháp. Không có đoạn mã hoàn chỉnh nào ở đây.

Mục lục

1 Vì sao là ba kỹ thuật này? 2

2 Kỹ thuật 1 — CLAHE: Cân bằng lược đồ xám thích nghi có giới hạn tương phản 3
2.1 Xuất phát điểm: cân bằng lược đồ xám toàn cục . . . 3
2.2 Hai chỗ HE toàn cục thất bại . . . 3
2.3 Giải pháp cho thất bại 2: cắt và phân phối lại lược đồ . . . 3
2.4 Giải pháp cho thất bại 1: chia ô và nội suy bảng tra . . . 4
2.5 Đặc tả cài đặt . . . 5
2.6 Thí nghiệm minh họa . . . 5

3 Kỹ thuật 2 — Ngưỡng thích nghi bằng ảnh tích phân 6
3.1 Ảnh tích phân: định nghĩa và truy vấn $O(1)$ . . . 6
3.2 Ngưỡng Bradley–Roth . . . 6
3.3 Mở rộng Sauvola: thêm phương sai cục bộ . . . 7
3.4 Điều đáng chú ý nhất: độ phức tạp không phụ thuộc cửa sổ . . . 7
3.5 Đặc tả cài đặt . . . 8
3.6 Thí nghiệm minh họa . . . 8

4 Kỹ thuật 3 — Biến đổi Hough đa tỉ lệ 9
4.1 Nhắc lại và đặt vấn đề . . . 9
4.2 Vì sao không tồn tại một độ phân giải đúng . . . 9
4.3 Giải pháp 1: thô đến mịn (coarse-to-fine) . . . 9
4.4 Giải pháp 2: bỏ phiếu theo hướng gradient . . . 10
4.5 Giải pháp 3: mảng tích lũy cũng là một ảnh . . . 10
4.6 Đặc tả cài đặt . . . 11
4.7 Thí nghiệm minh họa . . . 11

5 Ba kỹ thuật, một mô thức 13
5.1 Quy định về việc dùng công cụ AI . . . 13

Tài liệu tham khảo gợi ý 14

1 Vì sao là ba kỹ thuật này?

Ba kỹ thuật trình bày trong tài liệu này là ba bài tập cài đặt độc lập. Chúng không phụ thuộc lẫn nhau và có thể làm theo thứ tự bất kỳ.

Điểm chung của cả ba là mỗi kỹ thuật *phá vỡ một ranh giới* mà bài giảng đã dựng lên:

- **CLAHE** bắt đầu như một toán tử điểm (bảng tra LUT), nhưng khi bắt nó thích nghi theo vùng, nó *biến thành* một toán tử lân cận. Ranh giới "điểm và lân cận là hai loại khác nhau" sụp đổ.
- **Ngưỡng thích nghi** là một toán tử điểm mà *tham số* của nó đến từ một toán tử lân cận. Nhờ ảnh tích phân, toán tử lân cận này chạy trong $O(1)$ mỗi điểm ảnh, bất kể cửa sổ to bao nhiêu. Ranh giới "cửa sổ càng lớn càng chậm" sụp đổ.
- **Hough đa tỉ lệ** nhắc ta rằng mảng tích lũy Hough *cũng là một ảnh*. Mọi toán tử các em viết ở Phần A đều dùng lại được trên nó: làm mượt bằng tích chập, tìm đỉnh bằng lọc cực đại địa phương, lọc nhiễu bằng ngưỡng. Ranh giới "ảnh và không gian tham số là hai thế giới khác nhau" sụp đổ.

Một câu hỏi xuyên suốt tài liệu

Cả ba kỹ thuật đều là câu trả lời cho cùng một câu hỏi: *làm sao để một quyết định toàn cục (một ngưỡng, một bảng tra, một ô lượng tử hóa) trở nên thích nghi với vị trí, mà không phải trả giá bằng thời gian tính toán?* Ba câu trả lời khác nhau cho cùng một câu hỏi.

Quy ước ký hiệu dùng chung

Ký hiệu	Ý nghĩa
$I(x, y)$	Ảnh xám đầu vào, x là cột, y là hàng
H, W	Số hàng, số cột của ảnh
L	Số mức xám (thường $L = 256$)
$h[k]$	Lược đồ xám (histogram), $k = 0, \ldots, L-1$
$c[k]$	Lược đồ tích lũy (CDF) đã chuẩn hóa
$T[\cdot]$	Bảng tra (LUT), ánh xạ mức xám $\to$ mức xám
$S(x, y)$	Ảnh tích phân (summed-area table)
(ρ, θ)	Tham số đường thẳng trong không gian Hough
$A[\,i, j\,]$	Mảng tích lũy Hough

Quy ước chỉ số: trong toàn bộ tài liệu, ảnh được lưu dạng `numpy` với thứ tự `I[y, x]` (hàng trước, cột sau). Mọi công thức toán viết theo (x, y); khi cài đặt phải đảo lại. Đây là nguồn lỗi phổ biến nhất của bài tập này.

2 Kỹ thuật 1 — CLAHE: Cân bằng lược đồ xám thích nghi có giới hạn tương phản

2.1 Xuất phát điểm: cân bằng lược đồ xám toàn cục

Bài giảng đã trình bày cân bằng lược đồ xám (Histogram Equalization, HE). Nhắc lại ngắn gọn dưới dạng *toán tử điểm*: ta xây một bảng tra duy nhất cho toàn ảnh,

$$T[k] \;=\; \text{round}[(L-1)\cdot c[k]]\,, \qquad c[k] \;=\; \frac{1}{N}\sum_{j=0}^{k} h[j], \qquad N = H\cdot W, \tag{1}$$

rồi áp dụng $I_{\text{out}}(x,y) = T[I(x,y)]$.

Điểm mấu chốt các em cần giữ trong đầu suốt phần này: **toàn bộ HE nằm gọn trong một mảng 256 phần tử**. Ảnh đầu vào chỉ tham gia vào việc *đếm*. Mọi thứ ta sẽ làm tiếp theo đều là thao tác trên những mảng 256 phần tử ấy, chứ không phải trên ảnh.

2.2 Hai chỗ HE toàn cục thất bại

Thất bại 1 — ánh sáng không đều. Giả sử ảnh chụp một nhãn container: nửa trái nằm dưới nắng, nửa phải trong bóng. Lược đồ xám toàn cục có hai cụm. Một bảng tra duy nhất phải phục vụ đồng thời cả hai cụm, nên nó kéo giãn khoảng giữa (vốn gần như trống) và nén cả hai cụm lại. Kết quả: không vùng nào được cải thiện.

Thất bại 2 — khuếch đại nhiễu. Xét một vùng phẳng, gần như đồng nhất, chẳng hạn nền kim loại sạch. Trong vùng đó, toàn bộ điểm ảnh tập trung ở vài mức xám. CDF ở những mức ấy dốc đứng. Đạo hàm của bảng tra chính là độ dốc CDF, mà độ dốc CDF tại mức k tỉ lệ với $h[k]$:

$$\frac{\Delta T[k]}{\Delta k} \;=\; (L-1)\,\frac{h[k]}{N}. \tag{2}$$

Vậy **cột lược đồ càng cao thì độ tương phản tại mức xám đó càng bị kéo giãn**. Nhiễu ± 1 mức xám trong vùng phẳng sẽ bị biến thành chênh lệch hàng chục mức. Đây chính là lý do HE làm ảnh trông "rỗ".

Hai vấn đề, hai Giải pháp độc lập

Thất bại 1 là vấn đề *không gian*: cần bảng tra khác nhau ở vị trí khác nhau ⇒ chia ô (tiling), sinh ra AHE.
Thất bại 2 là vấn đề *biên độ*: cần chặn độ dốc bảng tra ⇒ giới hạn tương phản (contrast limiting), chữ "CL" trong CLAHE.
Hai Giải pháp hoàn toàn độc lập. Hãy cài đặt và kiểm thử riêng từng cái trước khi ghép.

2.3 Giải pháp cho thất bại 2: cắt và phân phối lại lược đồ

Theo (2), muốn chặn độ dốc bảng tra tại một giá trị tối đa $s_{\max}$, ta chỉ cần chặn chiều cao lược đồ. Đặt **ngưỡng cắt** (clip limit) β. Thuật toán:

1. Với mỗi k, nếu $h[k] > \beta$: ghi nhận phần dư $e_k = h[k] - \beta$ rồi đặt $h[k] \leftarrow \beta$.
2. Cộng tổng phần dư $E = \sum_k e_k$.

3. Phân phối E đều lại cho toàn bộ L mức: $h[k] \leftarrow h[k] + \lfloor E/L \rfloor$.

Việc phân phối lại là *bắt buộc*, không phải tùy chọn: nếu chỉ cắt mà không trả lại, tổng $\sum_k h[k]$ không còn bằng N, CDF không đạt tới 1, và bảng tra sẽ không phủ hết dải sáng.

Chọn β như thế nào. Nếu ta muốn độ dốc bảng tra không vượt quá $s_{\max}$ lần độ dốc của phép biến đổi đồng nhất, thì từ (2) với N_{tile} là số điểm trong một ô:

$$\beta \;=\; \frac{N_{\text{tile}}}{L}\left[1 + \frac{\alpha}{100}\,(s_{\max} - 1)\right], \qquad \alpha \in [0, 100]. \tag{3}$$

Khi $\alpha = 0$, $\beta = N_{\text{tile}}/L$: lược đồ bị san phẳng hoàn toàn, ảnh ra $\approx$ ảnh gốc. Khi $\alpha = 100$ và $s_{\max}$ lớn, không cột nào bị cắt: ta quay về AHE thuần túy. Hai giới hạn này là cách nhanh nhất để phát hiện lỗi trong phần cắt lược đồ.

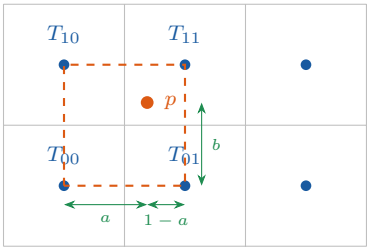
Trong thực tế, thư viện thường tham số hóa trực tiếp bằng $\texttt{clipLimit} = s_{\max}$. Các em có thể chọn cách nào cũng được, miễn là ghi rõ công thức đã dùng trong chú thích của hàm.

2.4 Giải pháp cho thất bại 1: chia ô và nội suy bảng tra

Chia ảnh thành lưới $G_y \times G_x$ ô (mặc định 8×8). Với mỗi ô, tính lược đồ, cắt theo (3), rồi dựng bảng tra riêng $T_{(i,j)}[\cdot]$ theo (1). Đến đây ta có $G_y \cdot G_x$ bảng tra, mỗi bảng 256 phần tử.

Nếu áp bảng tra của ô nào cho đúng ô ấy, kết quả sẽ có **đường ranh giới ô rõ rệt** — ảnh trông như bị chia ô vuông. Lý do: hai điểm ảnh cạnh nhau nhưng thuộc hai ô khác nhau được ánh xạ bằng hai hàm khác nhau.

Ý tưởng Coi mỗi bảng tra là một "mẫu" đặt tại *tâm ô*. Với một điểm ảnh bất kỳ, ta nội suy song tuyến giữa bốn bảng tra của bốn tâm ô lân cận.



Với $a, b \in [0, 1]$ là tọa độ tương đối của p trong hình chữ nhật nối bốn tâm ô, giá trị ra là

$$I_{\text{out}}(p) \;=\; (1-b)\Big[(1-a)\,T_{00}[v] + a\,T_{01}[v]\Big] \;+\; b\Big[(1-a)\,T_{10}[v] + a\,T_{11}[v]\Big], \qquad v = I(p). \tag{4}$$

Sai lầm nghiêm trọng nhất của phần này

Rất nhiều cài đặt sai theo kiểu: *áp từng bảng tra cho toàn ảnh, tạo ra $G_y \cdot G_x$ ảnh, rồi nội suy song tuyến giữa các ảnh đó*. Kết quả *tình cờ* gần đúng nhưng tốn bộ nhớ gấp 64 lần và sai ở biên. Phải nội suy **giá trị bảng tra**, tức bốn số vô hướng $T_{ij}[v]$, chứ không phải bốn ảnh. Đây là điểm chấm điểm cứng.

Xử lý biên. Điểm ảnh nằm ngoài lưới tâm ô (viền ngoài rộng nửa ô) không có đủ bốn hàng xóm:

- **Bốn góc ảnh:** chỉ có 1 bảng tra $\Rightarrow$ áp trực tiếp, không nội suy.
- **Bốn cạnh ảnh:** chỉ có 2 bảng tra $\Rightarrow$ nội suy tuyến tính 1 chiều.
- **Phần trong:** nội suy song tuyến đầy đủ theo (4).

Một mẹo cài đặt gọn: thay vì viết ba nhánh, hãy *kẹp* (clamp) chỉ số ô về biên hợp lệ. Khi p nằm ngoài, chỉ số ô trái và phải trùng nhau, trọng số a tự động trở nên vô nghĩa nhưng kết quả vẫn đúng vì $T_{00} = T_{01}$. Cách này cho một nhánh duy nhất cho cả ba trường hợp.

2.5 Đặc tả cài đặt

Hàm phải nộp — Kỹ thuật 1

```
def clip_histogram(hist, clip_limit):
    """Cắt và phân phối lại lược đồ.
    hist: mảng (L,) số nguyên.  Trả về mảng (L,) đã cắt, tổng không đổi."""

def build_tile_luts(img, grid=(8, 8), clip_limit=2.0):
    """Trả về mảng (Gy, Gx, L) uint8: bảng tra của từng ô."""

def clahe(img, grid=(8, 8), clip_limit=2.0):
    """CLAHE đầy đủ. img: (H, W) uint8.  Trả về (H, W) uint8."""
```

Thư viện được dùng: `numpy` (kể cả `np.cumsum`, `np.bincount`, `np.histogram`), `matplotlib` để vẽ.
Cấm: `cv2.createCLAHE`, `cv2.equalizeHist`, `skimage.exposure.equalize_adapthist`, `scipy.ndimage.map_coordinates`. Chỉ được gọi các hàm này trong *ô đối chiếu* có ghi chú rõ.

2.6 Thí nghiệm minh họa

Yêu cầu — Kỹ thuật 1

Áp dụng cài đặt của mình lên một ảnh hoặc một bộ ảnh, và trình bày **cả hai** nội dung sau trong notebook:

(a) Ảnh hưởng của tham số. Quét `clip_limit` và `grid` qua ít nhất bốn giá trị mỗi tham số, trình bày dạng lưới ảnh. Hãy chọn ảnh sao cho hiệu ứng *nhìn thấy được*: `clip_limit` chỉ lộ tác dụng trên ảnh có vùng phẳng nhiều nhiễu; `grid` chỉ lộ tác dụng trên ảnh có ánh sáng không đều theo vùng.

(b) So sánh với thuật toán đã học. Đặt cạnh nhau, trên cùng một ảnh: kéo giãn tương phản tuyến tính, hiệu chỉnh gamma, cân bằng lược đồ xám toàn cục, và CLAHE. Kèm lược đồ xám của từng kết quả. Chỉ ra một ảnh mà CLAHE thắng rõ rệt, và — quan trọng hơn — **một ảnh mà CLAHE thua** một trong ba cách kia.

3 Kỹ thuật 2 — Ngưỡng thích nghi bằng ảnh tích phân

3.1 Ảnh tích phân: định nghĩa và truy vấn $O(1)$

Ảnh tích phân (summed-area table) S của ảnh I được định nghĩa

$$S(x, y) \;=\; \sum_{x' \le x} \sum_{y' \le y} I(x', y'). \tag{5}$$

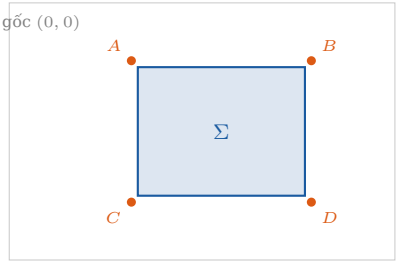
Tính trực tiếp theo định nghĩa mất $O(HW \cdot HW)$. Nhưng có một truy hồi hai chiều:

$$S(x, y) \;=\; I(x, y) + S(x-1, y) + S(x, y-1) - S(x-1, y-1), \tag{6}$$

cho phép dựng toàn bộ bảng trong *một lượt quét*, $O(HW)$. Số hạng trừ bù cho phần chồng lấn đã cộng hai lần — đây chính là nguyên lý bao hàm–loại trừ.

Truy vấn tổng chữ nhật. Với hình chữ nhật có góc trên-trái (x_1, y_1) và góc dưới-phải (x_2, y_2) (bao gồm cả biên):

$$\Sigma \;=\; S(x_2, y_2) - S(x_1 - 1, y_2) - S(x_2, y_1 - 1) + S(x_1 - 1, y_1 - 1). \tag{7}$$



$\Sigma = D - B - C + A$

(bốn lần truy cập bộ nhớ, độc lập kích thước cửa sổ)

Đệm viền là bắt buộc

Công thức (7) truy cập chỉ số -1. **Đừng** xử lý bằng lệnh `if`. Hãy dựng S với kích thước $(H+1) \times (W+1)$, hàng 0 và cột 0 toàn số không. Khi đó công thức trở thành thuần chỉ số dương và toàn bộ hàm truy vấn vectơ hóa được bằng lát cắt `numpy`.

Tràn số

Với ảnh `uint8` kích thước 2000×3000, tổng lớn nhất là $255 \cdot 6 \times 10^6 \approx 1.5 \times 10^9$ — vượt `int32` khi cộng bình phương. Luôn ép kiểu sang `np.int64` hoặc `np.float64` *trước* khi `cumsum`. Bảng tích phân của *bình phương* tràn `int32` rất nhanh: $255^2 \cdot 6 \times 10^6 \approx 3.9 \times 10^{11}$.

3.2 Ngưỡng Bradley–Roth

Ý tưởng: thay vì một ngưỡng toàn cục T, dùng ngưỡng phụ thuộc vị trí, lấy từ trung bình cục bộ trong cửa sổ $w \times w$ quanh điểm ảnh:

$$T(x, y) \;=\; m(x, y) \cdot (1 - t)\,, \qquad m(x, y) = \frac{1}{w^2}\,\Sigma_w(x, y), \tag{8}$$

với $t \approx 0.15$ là hệ số lệch. Điểm ảnh được gán nền trước (foreground) nếu $I(x, y) < T(x, y)$.

Diễn giải: mỗi điểm ảnh được so sánh với *hàng xóm của chính nó*, không phải với toàn ảnh. Vùng tối sẽ có m nhỏ, nên ngưỡng cũng nhỏ; chữ tối trên nền tối vẫn tách được. Đây chính xác là điều Otsu toàn cục không làm được.

Đây là toán tử gì?

Phép so sánh $I < T$ là toán tử điểm. Nhưng T đến từ trung bình cửa sổ, tức toán tử lân cận. Vậy ngưỡng thích nghi là *một toán tử điểm có tham số sinh bởi toán tử lân cận* — cùng một kiến trúc với bộ lọc song phương (trọng số = tích của nhân không gian và nhân giá trị) và với CLAHE (bảng tra = hàm của lược đồ cục bộ). Ba ví dụ, một mô thức.

3.3 Mở rộng Sauvola: thêm phương sai cục bộ

Bradley–Roth thất bại ở vùng *hoàn toàn phẳng*: nếu cửa sổ chỉ chứa nền, trung bình $\approx$ giá trị nền, và ngưỡng $m(1-t)$ nằm ngay dưới nó, nên nhiễu bị nhị phân hóa thành các đốm. Sauvola sửa lỗi này bằng cách đưa *độ lệch chuẩn cục bộ* $s(x,y)$ vào:

$$T(x,y) \;=\; m(x,y)\left[1+\kappa\left(\frac{s(x,y)}{R}-1\right)\right], \qquad \kappa \approx 0.34,\; R=128. \tag{9}$$

Khi $s \to 0$ (vùng phẳng), $T \to m(1-\kappa)$, hạ ngưỡng xuống *thấp hơn* trung bình khá nhiều, nên nhiễu không vượt ngưỡng. Khi s lớn (có chữ), $T \to m$, giữ độ nhạy.

Để tính s trong $O(1)$, cần *hai* ảnh tích phân: S_1 của I và S_2 của I^2. Khi đó

$$s^2(x,y) \;=\; \frac{\Sigma_w^{(2)}}{w^2} - \left(\frac{\Sigma_w^{(1)}}{w^2}\right)^2. \tag{10}$$

Mất chính xác do triệt tiêu

Công thức (10) là hiệu của hai số lớn gần bằng nhau. Với ảnh gần đồng nhất, kết quả có thể ra *âm* do sai số dấu phẩy động, và `np.sqrt` trả về `nan`. Luôn kẹp: `var = np.maximum(var, 0.0)`. Dòng này trông thừa cho tới lần đầu tiên nó cứu các em khỏi một ảnh đầy `nan`.

3.4 Điều đáng chú ý nhất: độ phức tạp không phụ thuộc cửa sổ

Có ba cách tính trung bình cục bộ với bán kính cửa sổ r:

Cách làm	Chi phí / điểm ảnh	Ghi chú
Tích chập trực tiếp với nhân hộp	$O(r^2)$	Dùng chính hàm `convolve2d` đã viết
Tích chập tách được (hai nhân 1 chiều)	$O(r)$	Nhân hộp là tích của hai nhân 1 chiều
Ảnh tích phân	$O(1)$	Bốn lần truy cập bộ nhớ, bất kể r

Đo thời gian chạy của ba cách với $r \in \{1,2,4,8,16,32,64\}$ và vẽ trên thang log: hai đường đầu dốc lên, đường thứ ba *nằm ngang*. Đường nằm ngang đó là toàn bộ thông điệp của kỹ thuật này. Nếu đường ảnh tích phân của các em không nằm ngang, gần như chắc chắn còn sót một vòng lặp Python theo điểm ảnh ở đâu đó.

Câu hỏi nối sang Phần A

Ba bộ lọc hộp liên tiếp xấp xỉ rất tốt một Gauss (hệ quả của định lý giới hạn trung tâm). Hãy đo sai số của xấp xỉ đó so với Gauss thật, rồi thay bước làm

mượt trong Canny bằng nó. Sai số định vị biên thay đổi bao nhiêu? Ở giá trị σ nào thì sai số ngừng có ý nghĩa?

3.5 Đặc tả cài đặt

Hàm phải nộp — Kỹ thuật 2

```
def integral_image(img):
    """Trả về S kích thước (H+1, W+1), dtype int64, hàng/cột 0 bằng 0."""

def box_sum(S, y1, x1, y2, x2):
    """Tổng vùng [y1..y2, x1..x2] (bao gồm biên), vectơ hóa được."""

def local_mean_std(img, radius):
    """Trả về (mean, std), mỗi cái (H, W) float64. Dùng 2 ảnh tích phân."""

def adaptive_threshold(img, radius=15, method='bradley', t=0.15, k=0.34):
    """Trả về ảnh nhị phân (H, W) uint8 với giá trị 0 / 255."""
```

Thư viện được dùng: `numpy` (kể cả `np.cumsum`), `time`/`timeit`, `matplotlib`.
Cấm: `cv2.integral`, `cv2.adaptiveThreshold`, `cv2.boxFilter`, `cv2.blur`, `scipy.ndimage.uniform_filter`, `skimage.filters.threshold_sauvola`. Chỉ dùng trong ô đối chiếu.

3.6 Thí nghiệm minh họa

Yêu cầu — Kỹ thuật 2

(a) Ảnh hưởng của tham số. Quét `radius` qua ít nhất năm giá trị và `t` (hoặc `k`) qua ít nhất ba giá trị. Hai hiện tượng phải xuất hiện được trong lưới ảnh: `radius` quá nhỏ thì nét chữ dày bị rỗng ruột; `radius` quá lớn thì phương pháp thoái hóa dần về ngưỡng toàn cục. Chỉ ra vùng giá trị hợp lý và giải thích nó phụ thuộc vào cái gì trong ảnh.

(b) So sánh với thuật toán đã học. Trên cùng ảnh: ngưỡng cố định thủ công, ngưỡng Otsu toàn cục, Bradley–Roth, và Sauvola. Ảnh dùng để so sánh *phải* có ánh sáng không đều — trên ảnh chiếu sáng đều, cả bốn cho kết quả gần như nhau và phép so sánh trở nên vô nghĩa. Bổ sung một ảnh có vùng nền phẳng rộng để cho thấy chỗ Sauvola hơn hẳn Bradley–Roth.

(c) Đường cong thời gian. Ba đường theo bảng ở mục 3.4, trục y thang log.

4 Kỹ thuật 3 — Biến đổi Hough đa tỉ lệ

4.1 Nhắc lại và đặt vấn đề

Bài giảng đã trình bày Hough chuẩn tắc: mỗi điểm biên (x, y) bỏ phiếu cho mọi đường thẳng đi qua nó, tham số hóa bởi

$$\rho \;=\; x\cos\theta + y\sin\theta, \qquad \theta \in [0, \pi), \quad \rho \in [-D, D], \quad D = \sqrt{H^2 + W^2}. \tag{11}$$

Mảng tích lũy A được rời rạc hóa với bước $\Delta\rho$ và $\Delta\theta$.

Câu hỏi cốt lõi mà bài giảng bỏ ngỏ: **chọn $\Delta\rho, \Delta\theta$ bằng bao nhiêu?**

4.2 Vì sao không tồn tại một độ phân giải đúng

Xét một đoạn thẳng thật, dài ℓ điểm ảnh, tâm ở gốc tọa độ. Nếu ta ước lượng góc sai một lượng $\delta\theta$, thì đầu mút của đoạn bị dịch khỏi đường ước lượng một khoảng

$$d_{\max} \;\approx\; \frac{\ell}{2}\,\delta\theta. \tag{12}$$

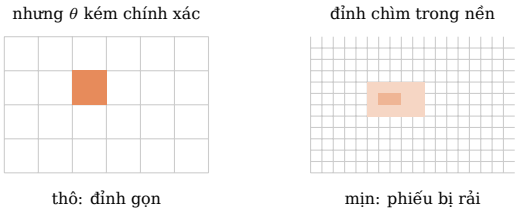
Để mọi điểm của đoạn vẫn rơi vào cùng một ô tích lũy, ta cần $d_{\max} \lesssim \Delta\rho$, tức

$$\boxed{\Delta\theta \;\lesssim\; \frac{2\,\Delta\rho}{\ell}} \tag{13}$$

Đọc (13) cho kỹ, vì nó là toàn bộ lý do của phần này:

- **Đoạn dài cần $\Delta\theta$ rất mịn.** Với $\ell = 500$ và $\Delta\rho = 1$, ta cần $\Delta\theta \lesssim 0.004$ rad $\approx 0.23°$.
- **Nhưng $\Delta\theta$ mịn thì đoạn ngắn bị vỡ phiếu.** Một đoạn 30 điểm ảnh có nhiễu vị trí ± 1 điểm sẽ rải phiếu ra nhiều ô θ lân cận, không ô nào đủ cao để nổi lên thành đỉnh.
- **$\Delta\theta$ thô thì đoạn dài bị lệch.** Phiếu của một đoạn dài bị chẻ ra hai ô ρ kề nhau, đỉnh tù đi và vị trí đỉnh sai.

Vậy *không có* một cặp $(\Delta\rho, \Delta\theta)$ tốt cho mọi đoạn thẳng trong cùng một ảnh. Đây không phải chuyện tinh chỉnh tham số; đây là một mâu thuẫn cấu trúc.



4.3 Giải pháp 1: thô đến mịn (coarse-to-fine)

Ý tưởng: đừng chọn một độ phân giải — dùng nhiều độ phân giải nối tiếp nhau.

1. Dựng mảng tích lũy *thô*, ví dụ $\Delta\theta_0 = 1°$, $\Delta\rho_0 = 2$ px. Ở độ phân giải này, đỉnh gọn và nổi rõ, dễ tìm.
2. Tìm K đỉnh ứng viên trong mảng thô.
3. Với mỗi ứng viên (ρ^*, θ^*), dựng lại một mảng tích lũy *nhỏ và mịn* chỉ phủ lân cận $[\rho^* \pm \Delta\rho_0,\ \theta^* \pm \Delta\theta_0]$, với bước chia nhỏ đi r lần (ví dụ $r = 8$). Chỉ những điểm biên đã bỏ phiếu cho ô thô mới tham gia bỏ phiếu lại.

4. Lặp bước 3 thêm 1–2 tầng nếu cần.

Chi phí: mảng mịn có kích thước cố định nhỏ (ví dụ 16×16) và số điểm bỏ phiếu đã được lọc, nên tổng chi phí xấp xỉ chi phí một lần Hough thô. Nhưng độ chính xác góc đạt được là $\Delta\theta_0 / r^{\,n}$ sau n tầng — tức tinh hơn nhiều lần so với việc dựng thẳng một mảng mịn, mà lại không bị vỡ phiếu.

Bẫy quấn vòng ở $\theta = 0$ và $\theta = \pi$

Khi mở cửa sổ lân cận quanh $\theta^* \approx 0$, phần âm phải quấn về phía π — **và kèm theo đổi dấu** ρ. Từ (11), đường thẳng (ρ, θ) và $(-\rho, \theta + \pi)$ là *cùng một đường*. Cài đặt bỏ qua chi tiết này sẽ mất mọi đường gần thẳng đứng — đúng lúc mà mọi ảnh chụp chứng từ đều có cạnh gần thẳng đứng. Hãy viết một bài kiểm thử riêng cho trường hợp $\theta \in \{0.5^\circ,\ 179.5^\circ\}$.

4.4 Giải pháp 2: bỏ phiếu theo hướng gradient

Hough chuẩn tắc lãng phí: một điểm biên bỏ phiếu cho *tất cả* N_θ giá trị góc, dù ta đã biết hướng của nó. Nhưng Canny (bước 1) đã tính sẵn gradient (g_x, g_y) tại mọi điểm biên. Pháp tuyến của đường thẳng đi qua điểm đó chính là hướng gradient:

$$\theta_g \;=\; \operatorname{atan2}(g_y,\, g_x) \bmod \pi. \tag{14}$$

Vậy chỉ cần bỏ phiếu cho $\theta \in [\theta_g - \delta,\ \theta_g + \delta]$, với δ khoảng 5°–15° để dung sai nhiễu gradient. Số phiếu mỗi điểm giảm từ N_θ xuống còn $2\delta / \Delta\theta$ — thường là giảm 10–20 lần.

Lợi ích không chỉ là tốc độ: nền của mảng tích lũy *sạch hẳn*, vì phiếu rác (phiếu cho những góc vô lý) biến mất. Tỉ số đỉnh trên nền tăng mạnh, và đó mới là điều làm bước tìm đỉnh trở nên dễ dàng.

Nhập nhằng dấu

θ_g và $\theta_g + \pi$ cho cùng một đường thẳng nhưng khác dấu ρ. Sau khi lấy mod π, phải bảo đảm ρ tính theo (11) vẫn nhất quán. Cách an toàn: luôn tính ρ trực tiếp từ (x, y, θ) sau khi đã chuẩn hóa θ về $[0, \pi)$, đừng suy ngược dấu.

4.5 Giải pháp 3: mảng tích lũy cũng là một ảnh

Đây là chỗ Phần A quay lại đầy đủ. Sau khi bỏ phiếu, A là một mảng 2 chiều — tức là một ảnh. Mọi toán tử các em đã viết đều áp dụng được:

Vấn đề trong A	**Toán tử Phần A**	**Vì sao hiệu quả**
Phiếu bị chẻ sang ô kề do lượng tử hóa	Tích chập với nhân Gauss nhỏ ($\sigma \approx 1$ ô)	Gộp lại phiếu của cùng một đường thẳng thật
Một đường thẳng sinh ra chùm đỉnh sát nhau	Lọc cực đại địa phương (max filter) $\rightarrow$ triệt phi cực đại	Chỉ giữ đỉnh trội trong bán kính cho trước
Nền phiếu rác	Ngưỡng — có thể là ngưỡng *thích nghi* (Kỹ thuật 2!)	Nền Hough không đồng đều theo ρ: ô ρ gần 0 nhận nhiều phiếu hơn

Một quan sát dễ bỏ qua

Nền của mảng tích lũy *không* đồng đều. Với ảnh chữ nhật, một đường thẳng đi gần tâm cắt qua ảnh với chiều dài lớn hơn đường thẳng đi gần góc, nên nhận được nhiều phiếu hơn *ngay cả khi cả hai đều không phải đường thẳng thật*. Vì vậy một ngưỡng toàn cục trên A thiên vị đường gần tâm. Ngưỡng thích nghi trên A — đúng hàm ở Kỹ thuật 2 — sửa được điều này.

4.6 Đặc tả cài đặt

Hàm phải nộp — Kỹ thuật 3

```
def hough_accumulate(edges, theta_range, rho_range, n_theta, n_rho,
                     grad_dir=None, delta_deg=None):
    """Bỏ phiếu vào mảng tích lũy. Nếu grad_dir khác None, chỉ bỏ phiếu
    trong dải +/- delta_deg quanh hướng gradient.
    Trả về (A, thetas, rhos)."""

def find_peaks(A, n_peaks, nms_radius=3, smooth_sigma=1.0):
    """Làm mượt A bằng hàm tích chập của chính nhóm, triệt phi cực đại,
    trả về danh sách (rho, theta, votes) sắp giảm dần."""

def hough_multiscale(edges, grad_dir, n_levels=2, refine_factor=8, ...):
    """Hough thô-đến-mịn. Trả về danh sách (rho, theta) đã tinh chỉnh."""
```

Thư viện được dùng: `numpy`, hàm `convolve2d` do chính nhóm viết ở Phần A, `matplotlib`.
Cấm: `cv2.HoughLines`, `cv2.HoughLinesP`, `skimage.transform.hough_line`, `skimage.transform.hough_line_peaks`, `scipy.signal.find_peaks` (áp cho mảng 2 chiều), `scipy.ndimage.maximum_filter`. Chỉ dùng trong ô đối chiếu.

4.7 Thí nghiệm minh họa

Yêu cầu — Kỹ thuật 3

Kỹ thuật này là kỹ thuật duy nhất trong ba kỹ thuật cho phép tạo **ảnh có đáp án đúng tuyệt đối**: các em tự vẽ đường thẳng với (ρ_0, θ_0) đã biết. Hãy tận dụng — phần lớn thí nghiệm dưới đây nên chạy trên ảnh tổng hợp, rồi mới kiểm lại trên ảnh thật.

(a) Ảnh hưởng của tham số. Quét $\Delta\theta$ và $\Delta\rho$ qua ít nhất bốn giá trị. Trình bày *cả* mảng tích lũy (dưới dạng ảnh) *và* các đường thẳng phát hiện được. Mảng tích lũy là nơi hiện tượng vỡ phiếu nhìn thấy rõ nhất. Sau đó quét số tầng n và hệ số tinh chỉnh r, vẽ sai số góc theo n.

(b) So sánh với thuật toán đã học. Trên cùng bộ ảnh: Hough chuẩn tắc ở độ phân giải thô, Hough chuẩn tắc ở độ phân giải mịn, và Hough đa tỉ lệ. So sánh theo ba chỉ số: sai số góc so với đáp án đúng, tỉ số đỉnh trên nền của mảng tích lũy, và thời gian chạy. Bổ sung một lần chạy bật/tắt bỏ phiếu theo hướng gradient trên cùng ảnh.

(c) Hai trường hợp bắt buộc có trong bộ ảnh: một ảnh chứa đường gần thẳng đứng (θ_0 quanh 0° hoặc 179°), và một ảnh chứa đồng thời đoạn thẳng rất dài và đoạn thẳng rất ngắn. Cả hai đều là chỗ Hough một tỉ lệ hỏng.

5 Ba kỹ thuật, một mô thức

Ba kỹ thuật trong tài liệu này được phát biểu bằng ba ngôn ngữ khác nhau — lược đồ xám, tổng tích lũy, không gian tham số — nhưng chúng là ba biến thể của cùng một ý tưởng:

Kỹ thuật	**Quyết định toàn cục bị thay thế**	**Thay bằng cái gì**
CLAHE	Một bảng tra duy nhất cho cả ảnh	Một bảng tra cho mỗi ô, nội suy song tuyến giữa chúng
Ngưỡng thích nghi	Một ngưỡng duy nhất cho cả ảnh	Một ngưỡng cho mỗi điểm ảnh, sinh từ thống kê cửa sổ tính trong $O(1)$
Hough đa tỉ lệ	Một cặp $(\Delta\rho, \Delta\theta)$ duy nhất	Nhiều độ phân giải nối tiếp, mỗi tầng chỉ tinh chỉnh vùng lân cận đỉnh của tầng trước

Trong cả ba trường hợp, cái giá phải trả lẽ ra là thời gian tính toán — và trong cả ba trường hợp, cái giá ấy được né bằng một thủ thuật khác nhau: nội suy thay vì tính lại, bảng tích lũy thay vì quét cửa sổ, thô-đến-mịn thay vì mịn ngay từ đầu.

Câu hỏi để tự trả lời

Cả ba kỹ thuật đều là *toán tử điểm có tham số sinh bởi toán tử lân cận*. Bộ lọc song phương ở Phần A cũng vậy (trọng số = tích của nhân không gian và nhân giá trị). Vậy còn phép biến đổi affine thì sao — có cách nào làm cho nó "thích nghi theo vị trí" theo đúng nghĩa này không, và khi đó nó còn là affine nữa không?

5.1 Quy định về việc dùng công cụ AI

Mỗi kỹ thuật gồm hai phần nộp, và quy định cho hai phần này **ngược nhau**.

	Phần cài đặt	**Phần thí nghiệm minh họa**
Nội dung	Các hàm trong ô "Hàm phải nộp"	Chọn ảnh, dựng bộ ảnh, quét tham số, vẽ hình, so sánh với thuật toán đã học
Công cụ AI	**Không được dùng** — kể cả để gợi ý, sửa lỗi hay giải thích mã	**Được dùng** — để sinh/chọn ảnh phù hợp và dựng khung thí nghiệm
Ràng buộc	Tự viết hoàn toàn, chỉ dùng thư viện trong danh sách cho phép	**Mọi prompt đã dùng phải được chép nguyên văn vào notebook**, ngay trên ô mã tương ứng. Notebook phải chạy từ đầu đến cuối.

Cách ghi prompt trong notebook

Trước mỗi ô mã có dùng AI hỗ trợ, chèn một ô markdown theo mẫu:

```
### Có dùng AI hỗ trợ
Công cụ: <tên công cụ, phiên bản nếu biết>
Mục đích: <sinh ảnh thử / dựng vòng lặp quét tham số / vẽ lưới ảnh / ...>
```

```
Prompt:
> <chép nguyên văn prompt đã dùng>

Đã sửa lại những gì: <mô tả ngắn phần các em phải sửa để chạy đúng>
```

Ô mã nào không có ô markdown này được hiểu là do các em tự viết. Thí nghiệm dùng AI mà không khai báo prompt sẽ không được tính điểm cho phần đó.

Vì sao chia như vậy. Phần cài đặt là chỗ duy nhất các em thực sự học được ba thuật toán này; nhờ người khác viết hộ thì phần học biến mất, và mã do AI sinh cho đúng ba bài này hỏng theo những kiểu rất đặc trưng, rất khó phát hiện nếu không hiểu thuật toán:

- **CLAHE:** gần như luôn nội suy giữa các *ảnh* thay vì giữa các *bảng tra*, và gần như luôn bỏ bước phân phối lại phần dư sau khi cắt.
- **Ảnh tích phân:** thường quên đệm viền, quên ép kiểu `int64`, và quên kẹp phương sai về không âm.
- **Hough:** gần như luôn bỏ qua bẫy quấn vòng ở $\theta = 0$ và $\theta = \pi$, nên im lặng đánh mất mọi đường gần thẳng đứng.

Cả ba lỗi trên đều cho ra ảnh *trông có vẻ đúng*.

Phần thí nghiệm thì ngược lại: chỗ khó không nằm ở việc viết mã vẽ hình, mà ở việc *nghĩ ra thí nghiệm nào đáng làm và chọn ảnh nào làm hiện ra hiệu ứng*. Một bộ ảnh chọn sai khiến cả bốn thuật toán cho kết quả như nhau và phép so sánh thành vô nghĩa — không công cụ nào chọn hộ các em được điều đó, vì nó đòi hỏi hiểu thuật toán sẽ hỏng ở đâu. Đó chính là phần được chấm.

Tài liệu tham khảo gợi ý

1. K. Zuiderveld, “Contrast Limited Adaptive Histogram Equalization,” in *Graphics Gems IV*, Academic Press, 1994, pp. 474–485. — Nguồn gốc của CLAHE, mô tả đầy đủ phần cắt lược đồ và nội suy.
2. S. M. Pizer et al., “Adaptive histogram equalization and its variations,” *Computer Vision, Graphics, and Image Processing*, vol. 39, no. 3, pp. 355–368, 1987.
3. F. C. Crow, “Summed-area tables for texture mapping,” *ACM SIGGRAPH Computer Graphics*, vol. 18, no. 3, pp. 207–212, 1984. — Bài báo giới thiệu ảnh tích phân.
4. D. Bradley and G. Roth, “Adaptive Thresholding Using the Integral Image,” *Journal of Graphics Tools*, vol. 12, no. 2, pp. 13–21, 2007.
5. J. Sauvola and M. Pietikäinen, “Adaptive document image binarization,” *Pattern Recognition*, vol. 33, no. 2, pp. 225–236, 2000.
6. J. Illingworth and J. Kittler, “The Adaptive Hough Transform,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 9, no. 5, pp. 690–698, 1987. — Nguồn gốc của ý tưởng thô-đến-mịn trong không gian tham số.
7. R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., Springer, 2022. — Mục 3.1 (toán tử điểm), 3.3 (toán tử lân cận), 3.6 (biến đổi hình học), 7.4 (Hough).